RapidFlow

USER GUIDE

ORFS Flow Runner

Install it, run a design from RTL to GDS and read the results, shown on real CVA6 runs, including stock against parallel abc_new

AUTHOR	rafay
DATE	2026-10-26
APP VERSION	1.0.1
DESIGN AND PLATFORM	cva6 on asap7, hierarchical
MACHINE	Intel Core i5-1235U, 8 threads, 10 GB RAM, Ubuntu 24.04

Contents

1. Summary **3**

2. At a glance **3**

3. Installing **3**

3.1 Build and install the package 3

3.2 First start 4

3.3 Choose the tools 4

4. The window **6**

5. Running a design **7**

6. Reading the results **9**

6.1 Reports → QoR checks 9

6.2 Reports → Report files 10

6.3 Logs 11

6.4 Debug 12

7. Worked example: stock and parallel abc_new **13**

7.1 What was compared 13

7.2 How to repeat it 14

7.3 Results 14

7.4 What the numbers mean 15

8. Other tasks **16**

8.1 Re-run from a stage 16

8.2 Runs that must outlive the app 16

8.3 Open a stage in the OpenROAD GUI 16

9. Troubleshooting **17**

10. Files **17**

1. Summary

ORFS Flow Runner is a desktop app for OpenROAD-flow-scripts (ORFS). It runs any design in an ORFS flow folder from RTL to GDS through the stock ORFS Makefile, so a run in the app gives the same result as the same make command in a terminal. This guide shows every part of the app with screenshots taken from real runs on this machine.

- **Install:** one Debian package, `orfs-flow-runner_1.0.1_all.deb`, puts the app under /opt with a menu entry. OpenROAD, Yosys and ORFS are installed separately; the app finds them on PATH or where Tools... points.
- **Run:** pick platform, design, config, CPU cores and a stage range from drop-down menus. The plan box shows the exact make command before anything runs.
- **Read results:** Reports shows pass/fail tiles and a metric-by-stage table; Logs and Report files have a line filter; Debug shows the environment, the resolved make variables and every warning ID.
- **Worked example:** CVA6 (asap7, hierarchical) synthesised twice from the app, once with stock Yosys and once with the parallel abc_new build. Synthesis took 47 min 55 s and 39 min 9 s; the two netlists are identical apart from the Yosys version line.

2. At a glance

Item	Value
App version	1.0.1 (gui/orfs_app.py)
Package	orfs-flow-runner_1.0.1_all.deb, built by gui/packaging/build_deb.sh
Needs	Python 3, GTK 3 with PyGObject, make; KLayout for the final GDS step
Installed separately	OpenROAD, Yosys, an OpenROAD-flow-scripts checkout
Tested on	Ubuntu 24.04, Intel Core i5-1235U (8 threads), 10 GB RAM
Tool versions in the screenshots	OpenROAD 26Q4-271-g0907a8656e; Yosys 0.69+274 (stock) and 0.69+280 (parallel abc_new); KLayout 0.28.16
Design in the screenshots	cva6 on asap7, hierarchical synthesis (config_hier*.mk)
Per-user files	~/.config/orfs-flow-runner, ~/.local/share/orfs-flow-runner/run_logs, ~/.cache/orfs-flow-runner

3. Installing

3.1 Build and install the package

From a checkout of RapidFlow, build the package and install it with apt. apt also installs GTK, PyGObject, make and KLayout when they are missing.

```
gui/packaging/build_deb.sh
sudo apt install ./dist/orfs-flow-runner_1.0.1_all.deb
```

The package installs the app in /opt/orfs-flow-runner, adds ORFS Flow Runner to the app menu and adds an orfs-flow-runner command. To update, build and install the newer package the same way. sudo apt remove orfs-flow-runner removes it and keeps your settings and run logs.

If you used gui/install_launcher.sh before, run gui/install_launcher.sh --uninstall first: its menu entry has the same name and would hide the installed one.

3.2 First start

Start ORFS Flow Runner from the app menu. It opens the flow folder you used last, else $ORFS_FLOW, else ~/OpenROAD-flow-scripts/flow. If it finds none it asks for one; pick the flow/ folder of your ORFS checkout. Flow folder... in the title bar changes it later.

3.3 Choose the tools

Tools... in the title bar sets which OpenROAD, Yosys and KLayout programs the runs use. Leave a row on Automatic to use the program on PATH, or the one in /usr/local/bin, /usr/bin, /opt/openroad/bin or ~/.local/bin. An app started from the menu has a short PATH, so this is how it finds tools installed elsewhere. Choices are saved in ~/.config/orfs-flow-runner/settings.json.

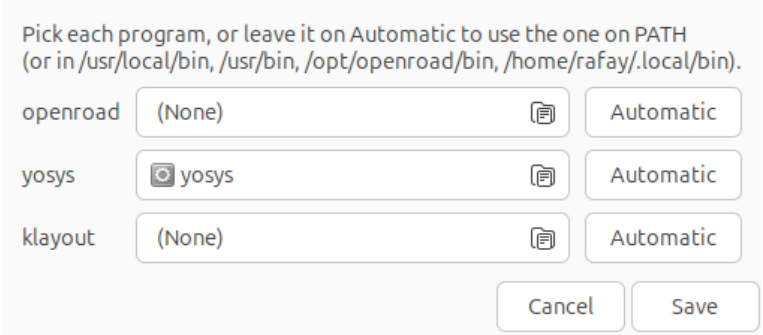


Figure 1. Tools... with Yosys set to the parallel abc_new build (yosys/build/yosys); OpenROAD and KLayout stay on Automatic.

Debug → Environment then shows the version and path of each tool, whether Yosys has the read_slang frontend that CVA6 needs, the CPU count, free memory and the run-log folder. Check it once after installing: a NOT FOUND row means a run would fail at once.

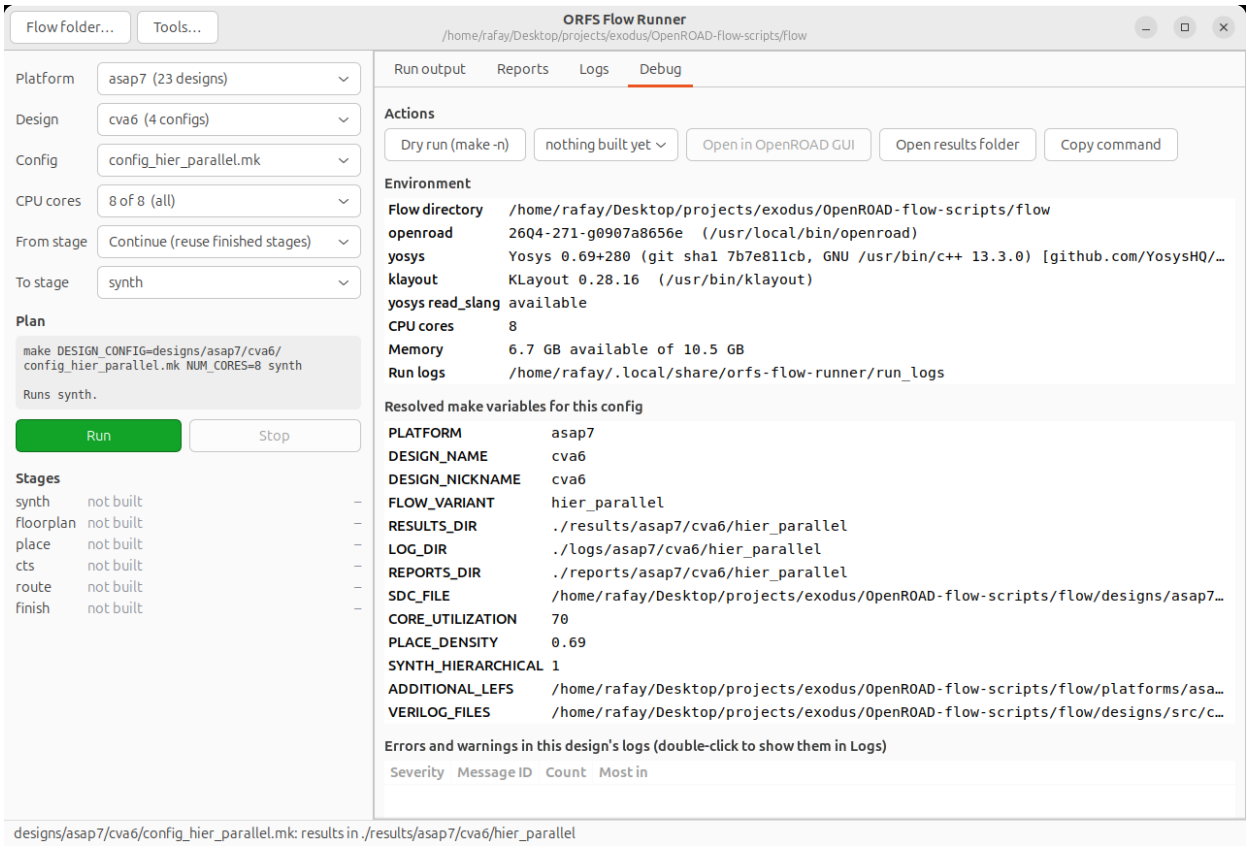


Figure 2. Debug → Environment after choosing the parallel Yosys: openroad, yosys and klayout found, read_slang available, logs in the per-user folder.

4. The window

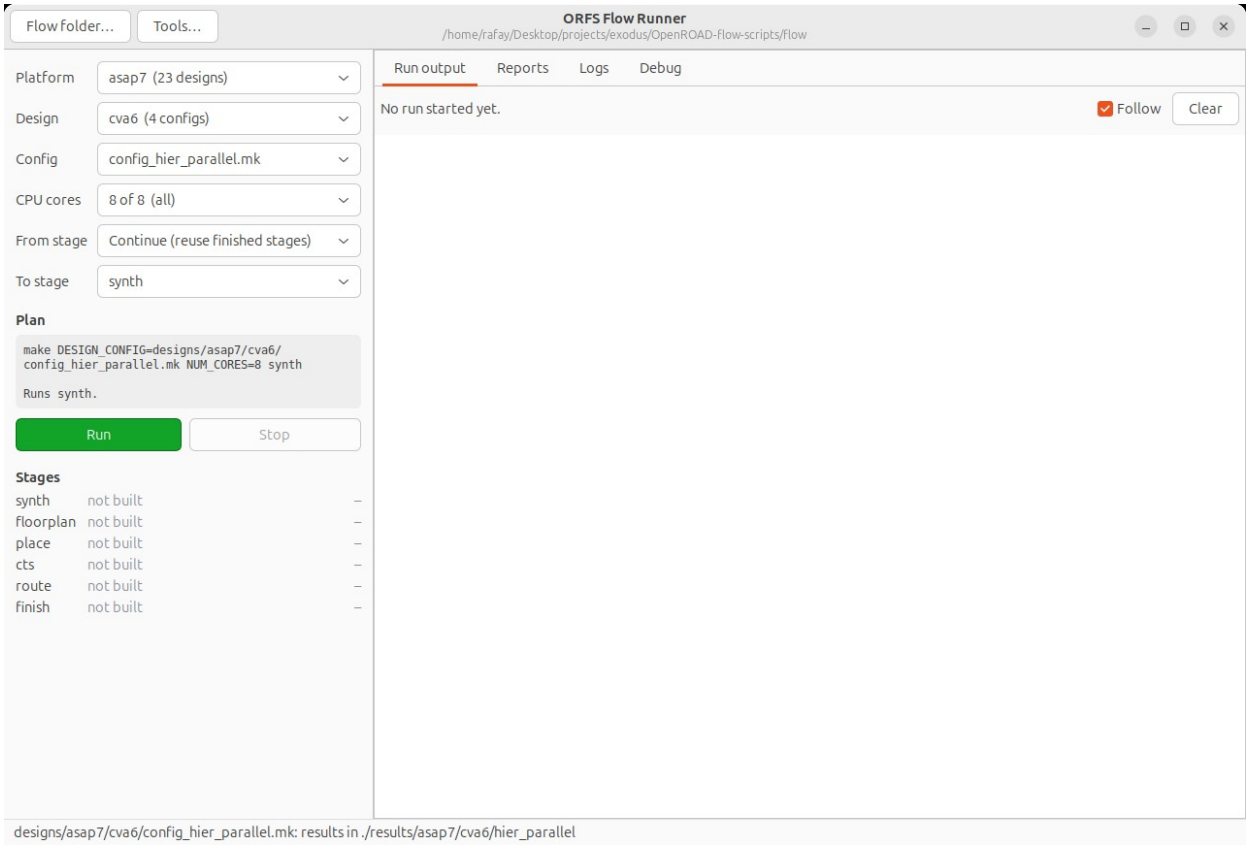


Figure 3. The window before a run: menus and plan on the left, output and results tabs on the right, status bar at the bottom.

Part	What it does
Title bar	Flow folder… and Tools…; the subtitle is the flow folder in use
Platform	every folder in flow/designs with designs; platforms whose PDK is missing are greyed out
Design, Config	the designs of that platform and their config*.mk files; the config sets DESIGN_CONFIG and with it the results folder
CPU cores	passed as NUM_CORES; fewer cores use less memory
From stage	Continue (reuse finished stages) or Re-run from a stage, which cleans that stage and every later one first
To stage	the last make target: synth, floorplan, place, cts, route or finish
Plan	the exact commands, the stages they run and anything they delete
Run, Stop	start the plan; Stop kills the running step and keeps finished stages
Stages	each stage as not built, running, partial, interrupted or done, with its runtime and peak memory
Tabs	Run output, Reports, Logs, Debug (sections 5 and 6)

Part	What it does
Status bar	while running: elapsed time, current step, tool, memory used and free; after: the result and the log path

The app remembers the flow folder, the menu choices and the tools for the next start.

5. Running a design

This walk-through synthesises CVA6 with hierarchical synthesis. Smaller designs such as nangate45/gcd work the same way and finish in minutes.

1. **Platform:** asap7. **Design:** cva6. **Config:** config_hier_parallel.mk (section 7 explains the A/B configs; config_hier.mk is the plain one).
2. **CPU cores:** 8 of 8. **From stage:** Continue. **To stage:** synth.
3. **Read the plan box.** It reads make DESIGN_CONFIG=designs/asap7/cva6/config_hier_parallel.mk NUM_CORES=8 synth and Runs synth.
4. **Press Run.** Run output streams the output: errors in red, warnings in amber, the run's own messages in blue. Untick Follow to scroll back without being pulled to the end.
5. **Watch Stages and the status bar.** synth shows running, and the status bar names the step (1_2_yosys), the tool and the memory in use.
6. **Wait for >> exit code 0.** The status bar then says Run finished after the elapsed time, with the path of the run log.

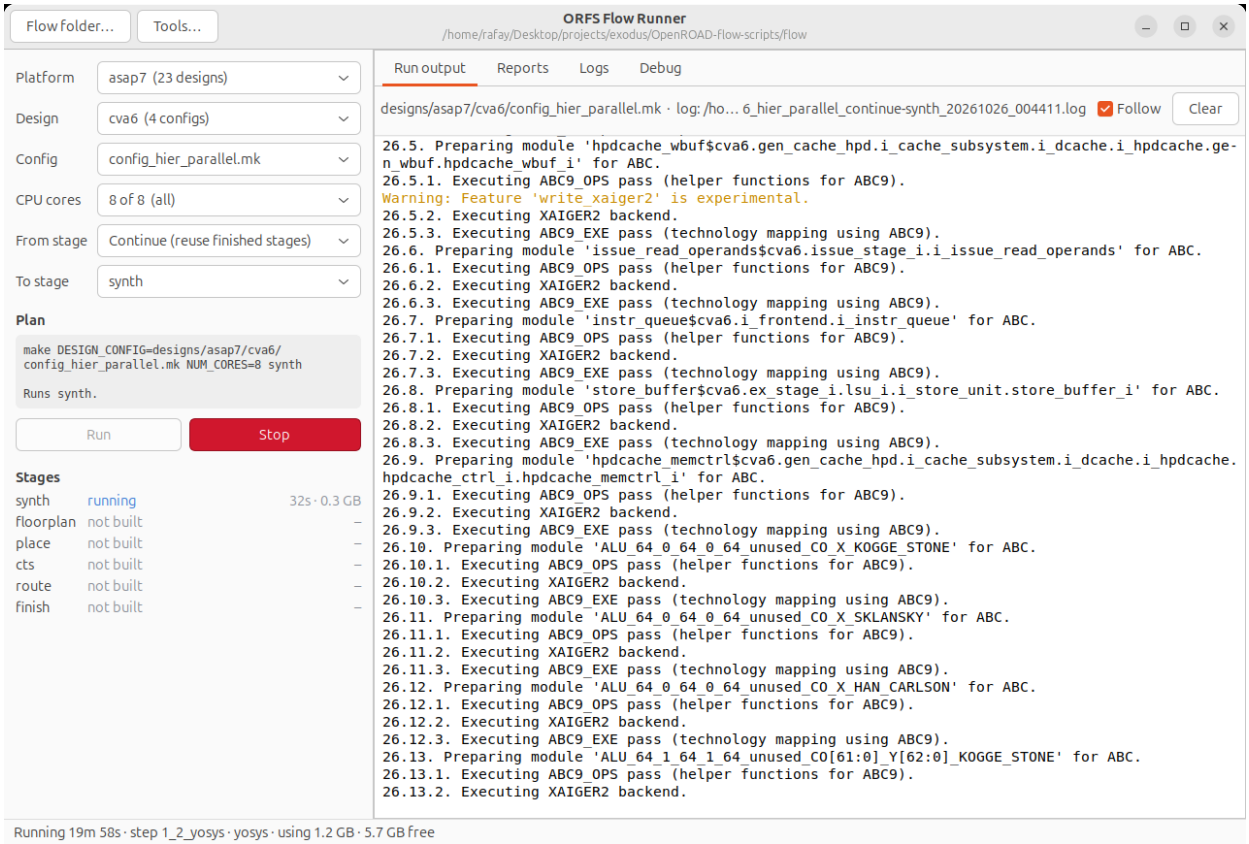


Figure 4. Twenty minutes into synthesis with the parallel Yosys: the step is 1_2_yosys and abc_new is preparing each module for ABC.

With stock Yosys the same point of the run looks different: abc_new maps one module at a time, so the output alternates Mapping module and ABC9 lines for each of the 93 modules (Figure 5).

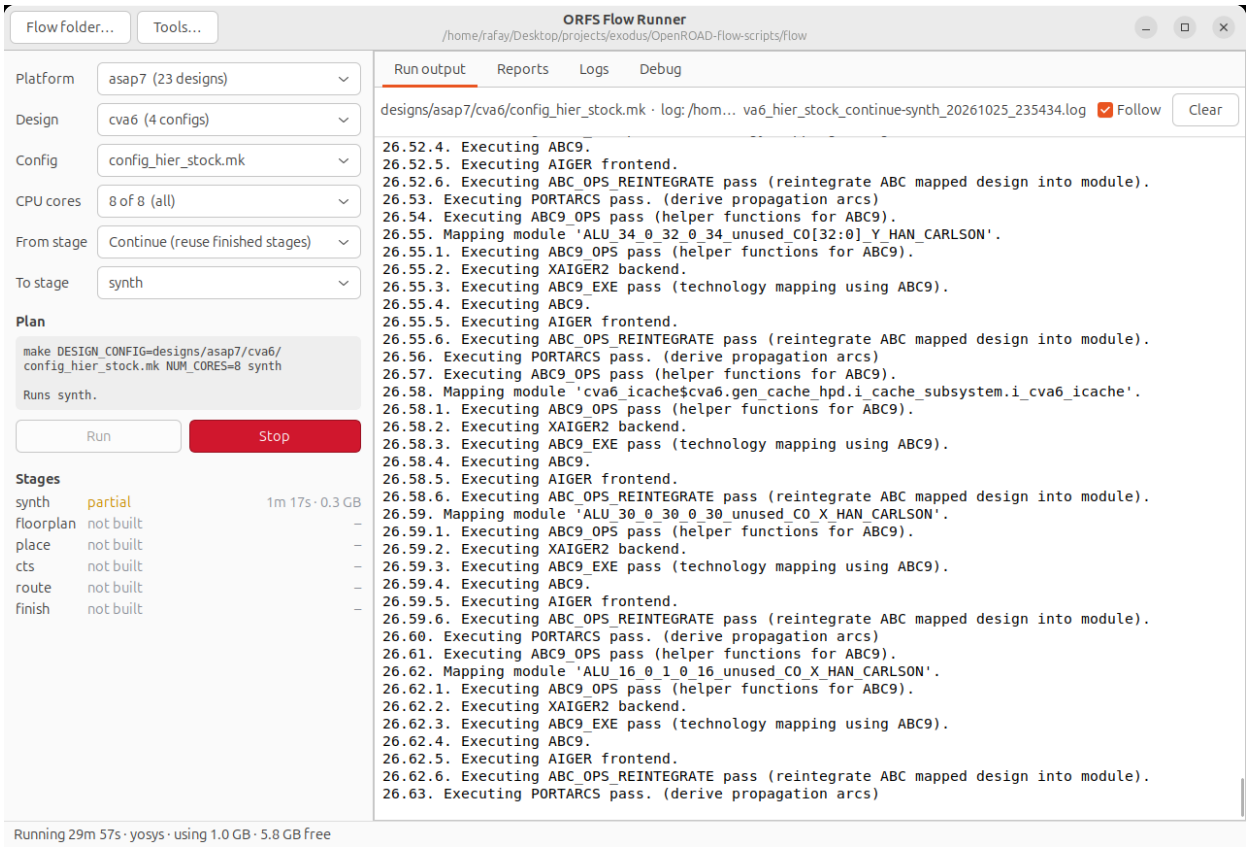


Figure 5. The same step with stock Yosys, 30 minutes in. Screenshot from app version 1.0.0, which showed synth as partial while it ran; 1.0.1 shows running.

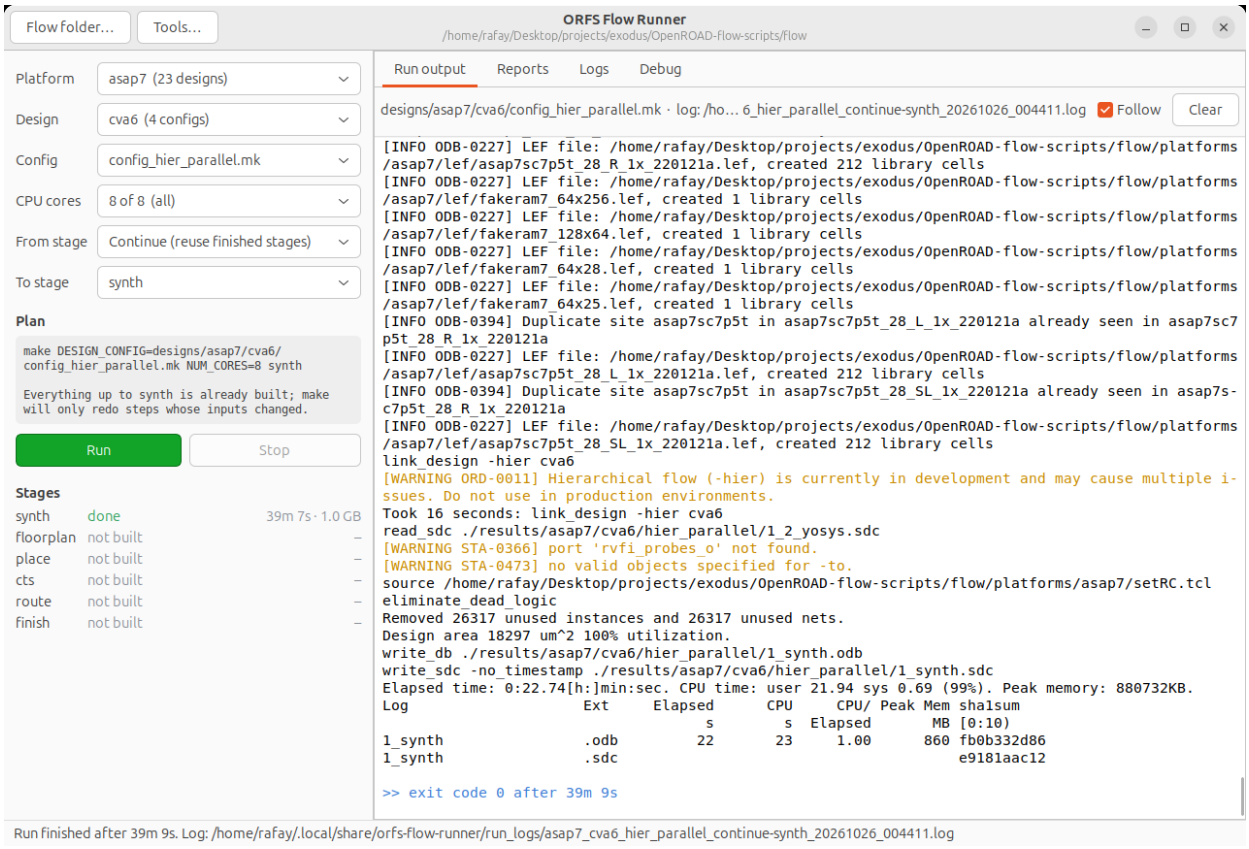


Figure 6. The finished run: synth done in 39 min 7 s with 1.0 GB peak, exit code 0, and the run log path in the status bar.

To stop a run, press Stop and confirm. The running step is killed, its partial log stays as a .tmp.log and finished stages are kept; the stage then shows interrupted. Closing the window during a run asks first.

6. Reading the results

6.1 Reports → QoR checks

QoR checks has pass/fail tiles for the latest stage that has metrics, then a table of every metric by stage. Red marks a failure: negative slack or a non-zero violation count. The numbers come from the ORFS metrics JSON files in logs/<platform>/<design>/<variant>/.

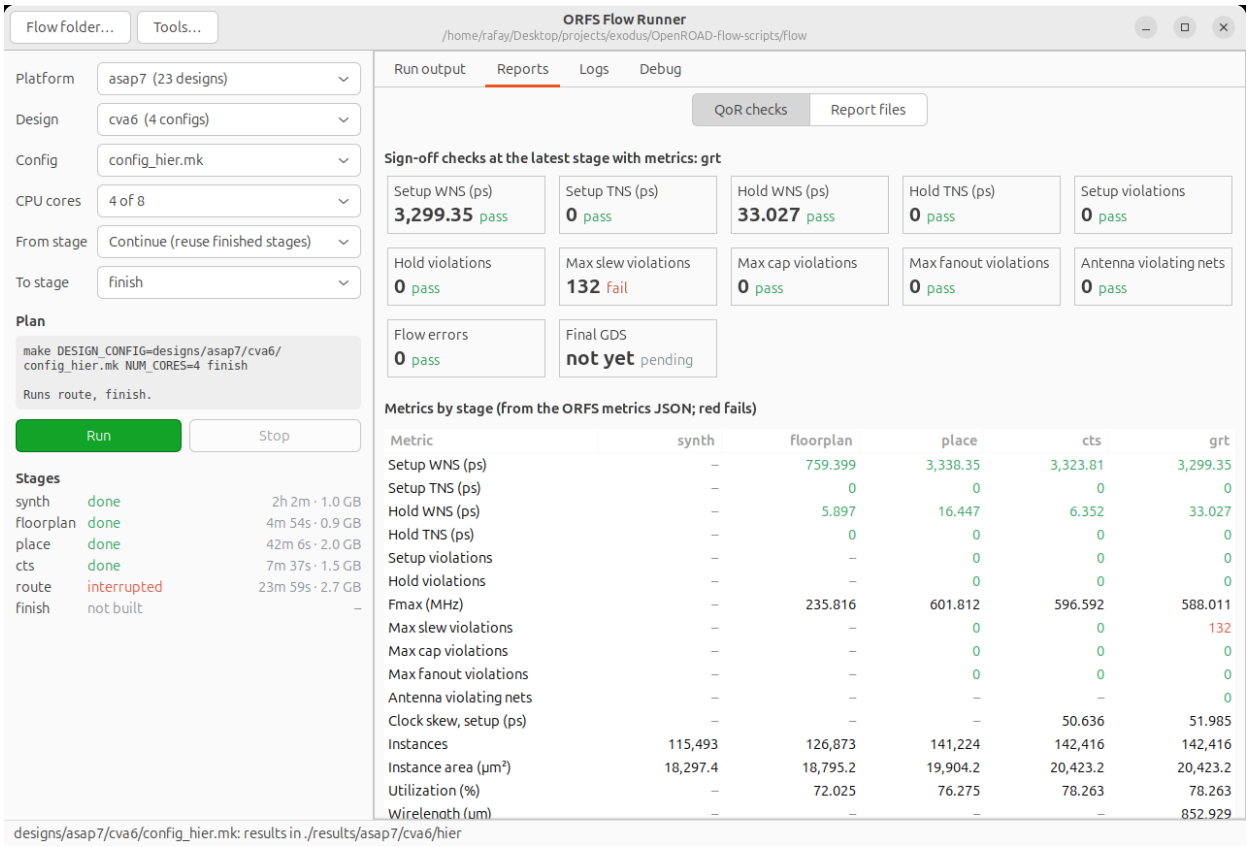


Figure 7. QoR checks for the earlier CVA6 hier run, which reached global routing before detailed routing was interrupted: timing passes, 132 max-slew violations fail.

After synthesis only, the table has one column: 115,493 instances and 18,297.4 μm² for CVA6, with 0 flow errors and 33 warnings. Both A/B runs show exactly these numbers.

6.2 Reports → Report files

Report files lists the design's reports folder. Pick a file to read it; type in the filter box to keep only matching lines, or tick Errors & warnings only.

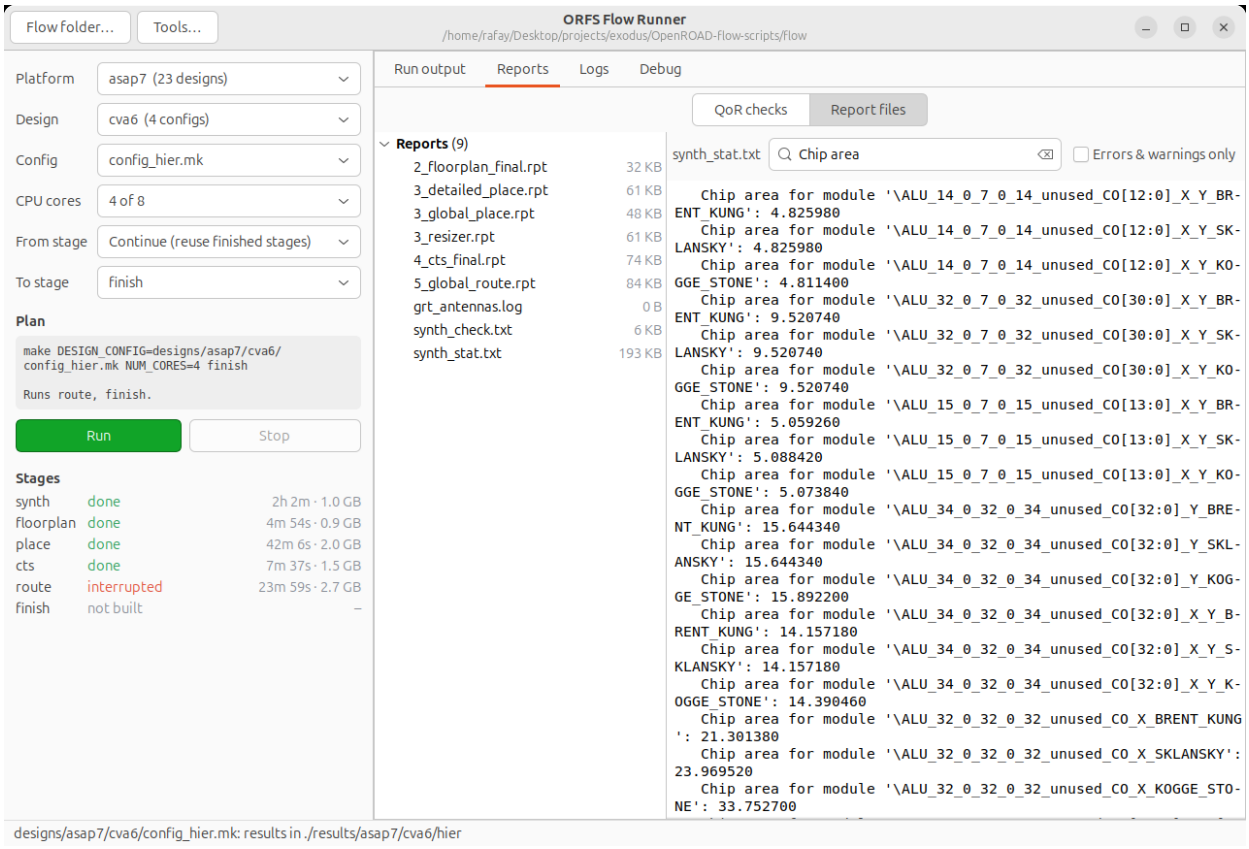


Figure 8. synth_stat.txt filtered to Chip area: the area of each synthesised module.

6.3 Logs

Logs lists the stage logs (one per step; a .tmp.log is a step that is running or was killed) and the run logs (one per run, newest first). The filter and Errors & warnings only work as in Report files. With Follow ticked, an open log that is still being written keeps scrolling to its end.

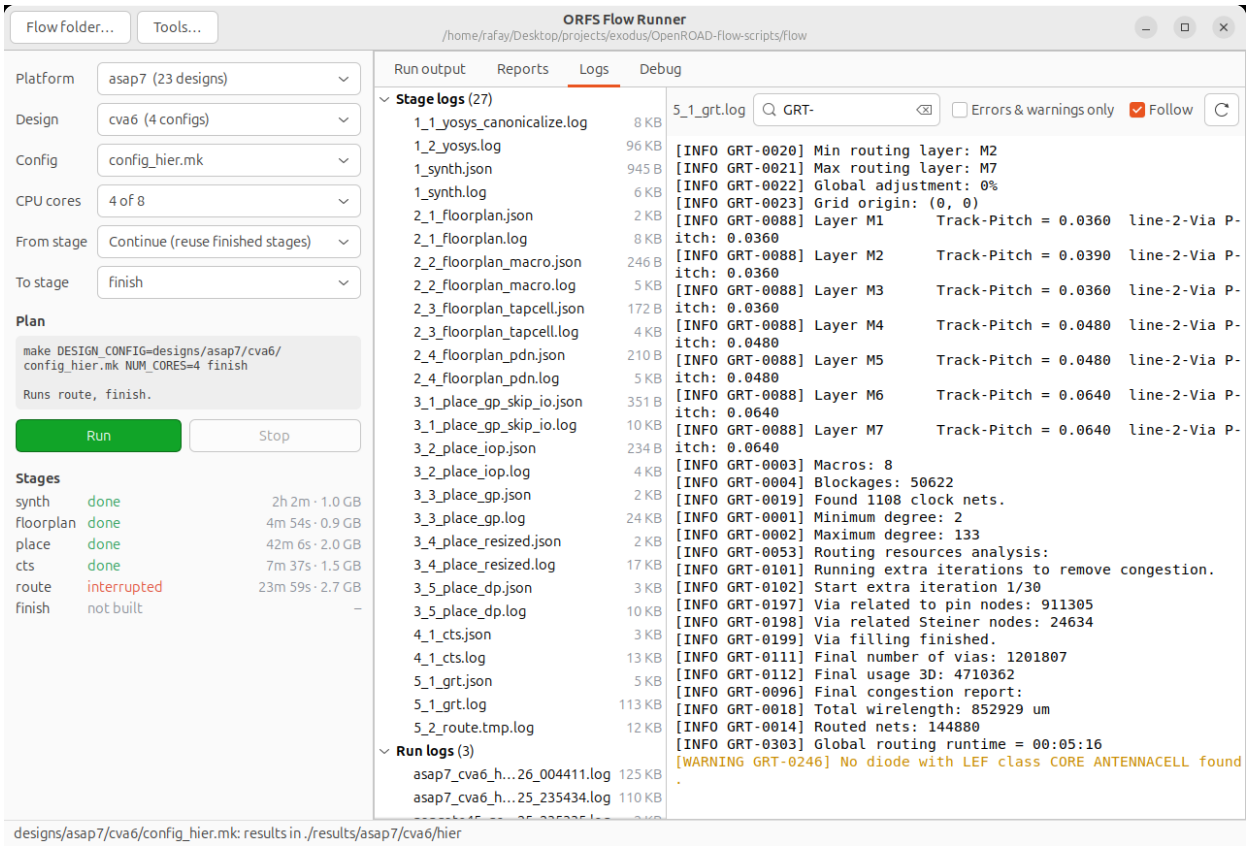


Figure 9. The global-route log filtered to GRT- messages; the run logs of the two A/B runs are at the bottom of the list.

6.4 Debug

Debug has four parts. Actions: Dry run (make -n) shows what make would run for the plan; Open in OpenROAD GUI opens a built stage; Open results folder; Copy command copies the plan for a terminal. Environment (Figure 2). Resolved make variables: PLATFORM, FLOW_VARIANT, RESULTS_DIR, SDC_FILE, CORE_UTILIZATION and others as make print-<VAR> reports them. Errors and warnings: a count of every message ID in the design's logs, errors first; double-click one to open its log in Logs, filtered to that ID.

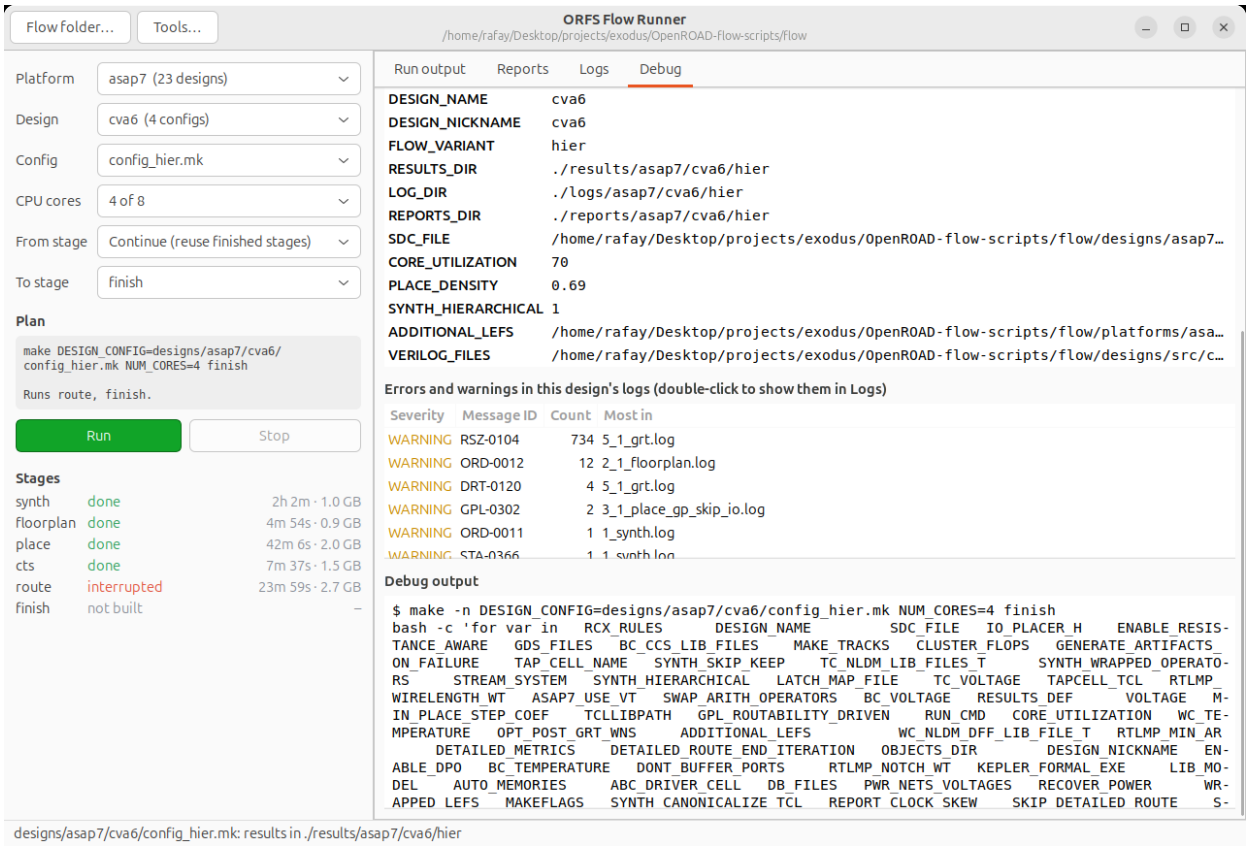


Figure 10. Debug scrolled down: the make variables, the warning IDs in the CVA6 logs (RSZ-0104 734 times, mostly in global routing) and a dry run.

7. Worked example: stock and parallel abc_new

7.1 What was compared

CVA6 synthesis spends most of its time in Yosys, and much of that in abc_new, which maps each of the 93 kept modules with ABC. The yosys/ submodule (PR #2) adds a parallel abc_new that maps independent modules at the same time. Two runs from the app measure what that saves on a full ORFS synthesis.

	Stock run	Parallel run
Config	config_hier_stock.mk	config_hier_parallel.mk
Results folder	results/asap7/cva6/hier_stock	results/asap7/cva6/hier_parallel
Yosys (Tools...)	/usr/local/bin/yosys, 0.69+274	yosys/build/yosys, 0.69+280 (0.69+274 plus parallel abc_new)
Settings	NUM_CORES=8, From Continue, To synth	same
Started	2026-10-25 23:54	2026-10-26 00:44, after the stock run

Both configs include config_hier.mk and change only FLOW_VARIANT, so the runs use identical settings and keep their results apart. The runs went one after the other on an otherwise idle

machine. While each ran, the number of yosys-abc processes and the machine's busy cores were sampled every 5 seconds.

```
# orfs/flow/designs/asap7/cva6/config_hier_parallel.mk
include $(dir $(lastword $(MAKEFILE_LIST)))config_hier.mk
export FLOW_VARIANT = hier_parallel
```

7.2 How to repeat it

1. Build Yosys from the yosys/ submodule (README, Step 2).
2. In the app choose asap7 / cva6 / config_hier_stock.mk, To stage synth, and set Yosys to the stock build in Tools…. Press Run.
3. When it finishes, choose config_hier_parallel.mk, set Yosys to yosys/build/yosys in Tools…, and press Run.
4. Compare the Stages times, the two run logs in Logs, and the Yosys log of each variant (1_2_yosys.log ends with Took … seconds: abc_new and the time per pass).

7.3 Results

Measure	Stock	Parallel	Change
Whole synthesis (app run time)	47 min 55 s (2,878 s)	39 min 9 s (2,352 s)	−526 s, 1.22×
1_1 canonicalize	77 s	32 s	file cache, not ABC
1_2_yosys step	2,771 s	2,291 s	−480 s, 1.21×
abc_new inside 1_2_yosys	1,496 s	1,150 s	−346 s, 1.30×
1_synth (OpenROAD)	21 s	23 s	none
Peak memory, Yosys	1,014 MB	1,049 MB	+35 MB
Most yosys-abc at once (5 s samples)	1	4	
Instances / area	115,493 / 18,297.4 μm²	115,493 / 18,297.4 μm²	identical
synth_stat.txt			identical
1_2_yosys.v netlist			identical except the Yosys version comment

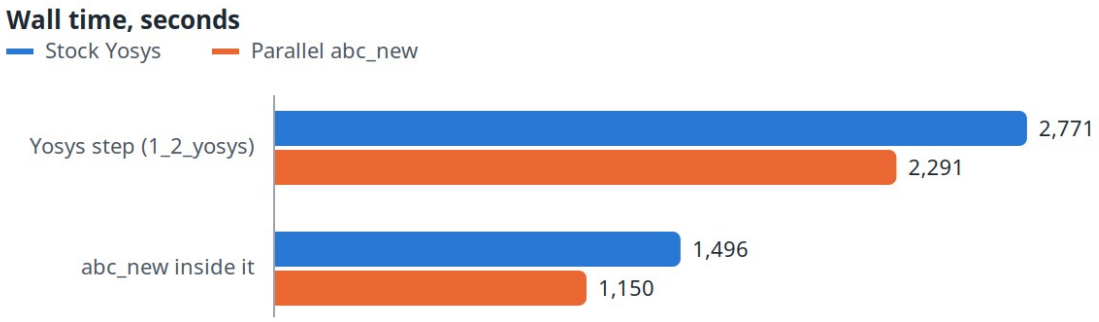


Figure 11. Wall time of the Yosys step and of abc_new inside it, one run each.

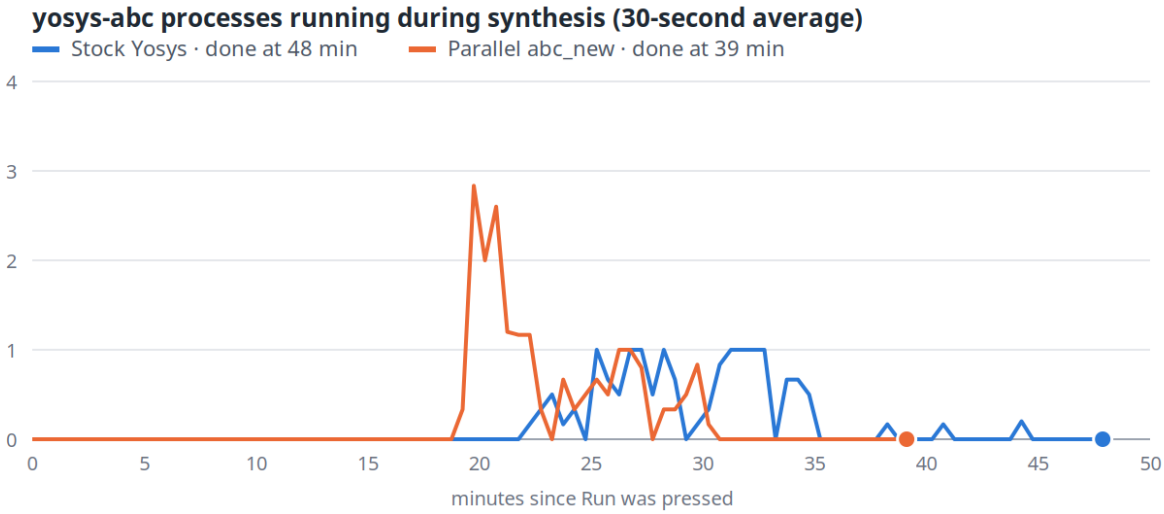


Figure 12. yosys-abc processes running over each run, averaged over 30 seconds. The parallel run starts mapping at about 19 minutes with up to 4 processes; the stock run never has more than one.

7.4 What the numbers mean

- The parallel build saves 346 s in abc_new and 526 s on the whole synthesis; 45 s of that is the canonicalize step, which read the RTL from a cold file cache in the first run and is unrelated to ABC.
- The mapping is unchanged: the stock and parallel netlists differ only in the comment that names the Yosys version, and the stock netlist is byte-identical to the earlier config_hier.mk run. The later stages of that run (Figure 7) therefore apply to both.
- Parallelism stays low: at most 4 ABC processes were seen at once, and 1.3 on average while any were running. Yosys still does serial work around each ABC call; in the parallel log write_xaiger2 alone takes 467 s over 186 calls. That serial work, not the cores, caps the saving.
- The 1.30× on abc_new is below the 1.47× measured in the abc_new change report, which replayed abc_new alone from a saved design. This run measures the full ORFS step, as a user would see it.

8. Other tasks

8.1 Re-run from a stage

Choose Re-run from <stage> in From stage. The plan lists the clean commands first, and Run asks for confirmation with the stages whose results, logs and reports will be deleted, partial ones included. Later stages are cleaned too, because they were built from the old results.

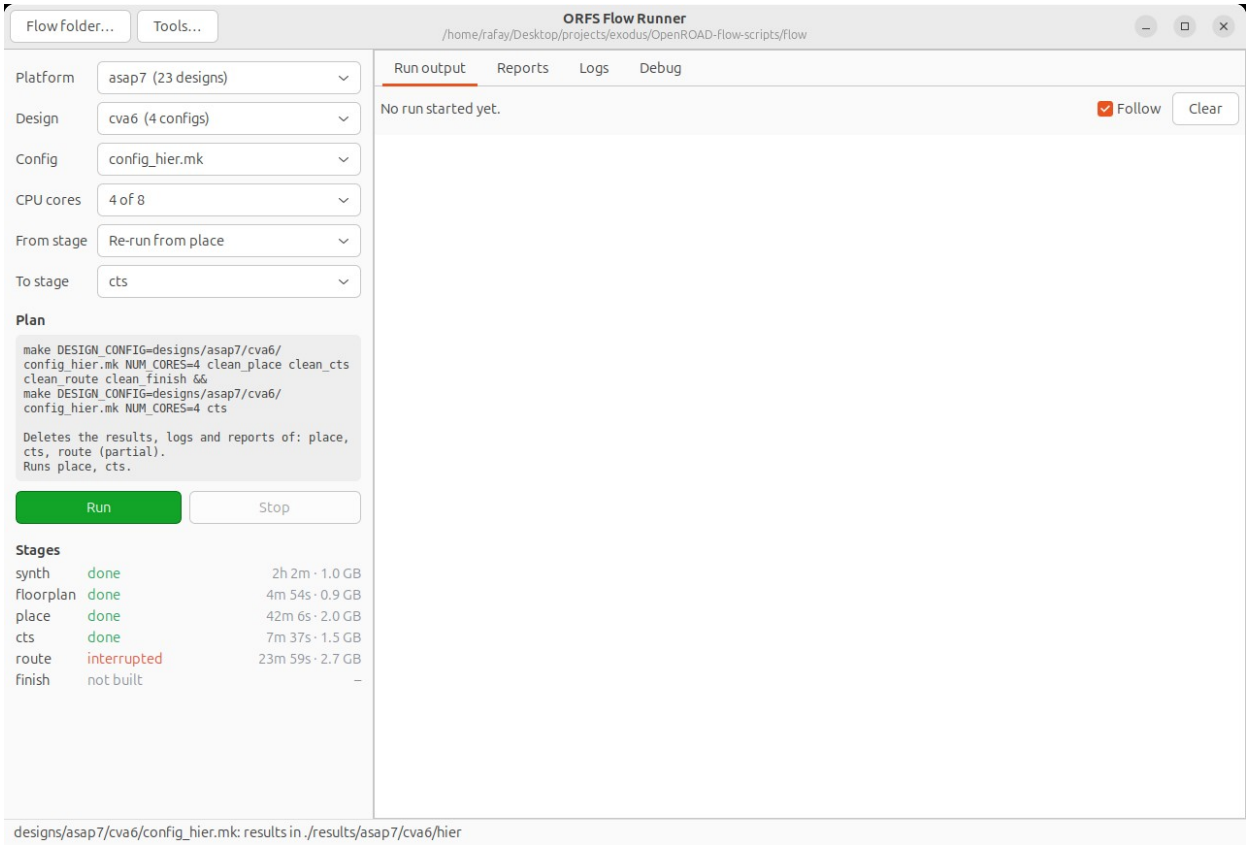


Figure 13. Re-run from place to cts on the earlier CVA6 run: the plan cleans place, cts and route, then runs place and cts.

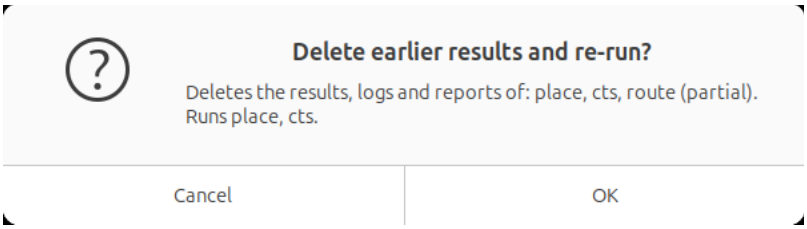


Figure 14. The confirmation before anything is deleted.

8.2 Runs that must outlive the app

Closing the app asks whether to stop its run. For a run that must survive logging out, use Debug → Copy command and start it with systemd-run --user as in the README's Running unattended section. The app still shows that run's results and logs, but not its live status.

8.3 Open a stage in the OpenROAD GUI

Debug → Actions: pick a built stage in the drop-down and click Open in OpenROAD GUI. Its output goes to ~/.cache/orfs-flow-runner/openroad_gui.log.

9. Troubleshooting

Problem	What to do
No module named gi at start	sudo apt install python3-gi gir1.2-gtk-3.0, and use the system python3, not a venv
A run fails at once with error 127	OpenROAD or Yosys was not found: check Debug → Environment and choose the program with Tools…
The Platform menu is empty	the folder is not an ORFS flow/ folder: use Flow folder…
A stage shows interrupted	the step was killed, usually for lack of memory; look for oom-kill in journalctl, use fewer cores, then run again with Continue
read_slang missing (CVA6)	build Yosys with the slang frontend (README, Step 1 and 2)
Make variables show make error	the config does not parse; the message is in Debug output
The installed app does not appear in the menu	an older gui/install_launcher.sh entry hides it: run gui/install_launcher.sh --uninstall

Limits: one run at a time per window; only runs started from the app show as running; viewers show the last 4 MB of a log, and the full output is always in the run log.

10. Files

Path	What it is
gui/orfs_app.py	the app
gui/flow_gui.py	the CVA6 web console; the app reuses its metric definitions
gui/packaging/build_deb.sh	builds the .deb; orfs-flow-runner.desktop and .svg are its menu entry and icon
gui/install_launcher.sh	menu and Desktop entry for running from a checkout, without the package
orfs/flow/designs/asap7/cva6/ config_hier_{stock,parallel}.mk	the A/B configs of section 7
~/.config/orfs-flow-runner/settings.json	flow folder, menu choices and tools
~/.local/share/orfs-flow-runner/ run_logs/	one log per run: <platform>_<design>_<variant>_<from>-<to>_<date>_<time>.log
~/.cache/orfs-flow-runner/	OpenROAD GUI log; safe to delete
<flow>/results|logs|reports/ <platform>/<design>/<variant>/	ORFS results (one .odb per stage, 6_final.gds), step logs with metrics JSON, and reports